

An underwater scene featuring a large, pinkish-purple jellyfish in the foreground, surrounded by various coral reefs and smaller jellyfish in the background. The water is a clear, vibrant blue.

# Operating Systems & System Software

## **Session 2.2**

### **PCB and 2 & 3 State Models**

**Mouli Sankaran**

## Session 2.2: Focus

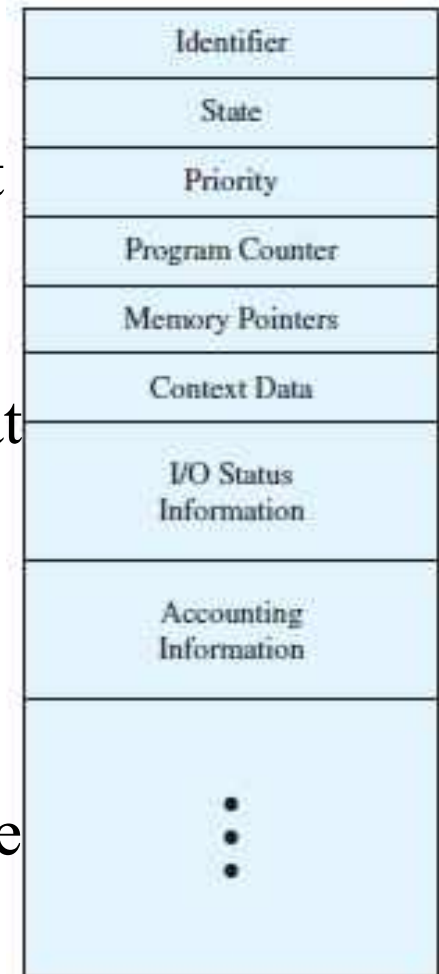
- **PCB: Process Control Block**
- **Process State Models**
  - **Two States Model**
    - **Dispatcher Queue**
    - **Dispatcher Queue Explained**
  - **Three States Model**
    - **Ready**
    - **Running**
    - **Blocked**



# Process Control Block

## Process Control Block (PCB)

- Process Control Block is a data structure created and managed by the OS for every process in the system.
- The significant point about the PCB is that it contains sufficient information so that it is possible to interrupt a running process and later resume its execution as if an interruption had never occurred.
- The process control block is the key data structure that enables the OS to support multiple processes to provide multiprocessing capability.
- When a process is interrupted, the current values of the program counter and the processor registers (context data) are saved in the appropriate fields of the corresponding PCB, maintained by the OS.
  - Or stored as part of the process area which is normally not done because it could potentially be corrupted being in user area.





## PCB Entries: Explained

|                        |
|------------------------|
| Identifier             |
| State                  |
| Priority               |
| Program Counter        |
| Memory Pointers        |
| Context Data           |
| I/O Status Information |
| Accounting Information |
| ⋮                      |

- **Identifier:** A unique identifier associated with this process, to distinguish it from all other processes.
- **State:** If the process is currently executing, it is in the **running state**.
- **Priority:** Priority level relative to other processes.
- **Program counter:** The address of the next instruction in the program to be executed, when the process is allotted CPU next.
- **Memory pointers:** Includes pointers to the program code and data associated with this process, plus any memory blocks shared with other processes.
- **Context data:** These are data that contained in various registers of the processor while the process was last executing on the CPU.
- **I/O status information:** Includes outstanding I/O requests, I/O devices (e.g.HDD) assigned to this process, a list of files in use by the process, and so on.
- **Accounting information:** Includes amount of processor time used by the process, files opened, any time limits, and so on.

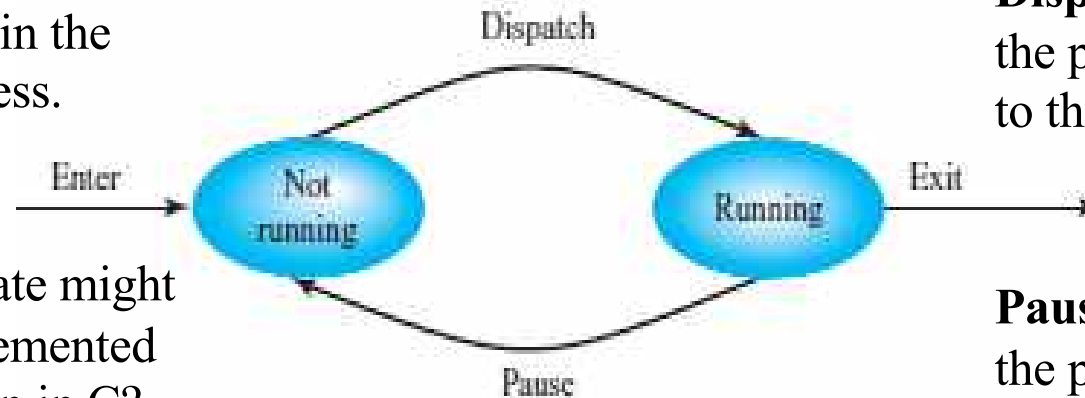


# Process State Models

# A Two States Process Model

## State Transition Diagram

**Note:** Recall that **state** is a **field** in the **PCB** of a process.



**Dispatch** means that the process is dispatched to the CPU for running.

How will the state might have been implemented in an OS, written in C?

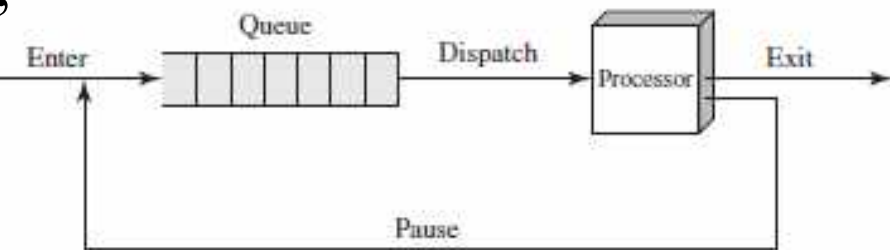
**ANS: enum**

**Pause** means that the process is stopped by taking it away from the CPU.

- We can construct the simplest possible model by observing that, at any time, a process is either being executed by a processor or not.
- In this model, a process may be in one of the **two states: Running or not Running**.
- This is the most simplistic process state but not useful because it does not capture the real-life states, a process goes through.

# OS: Dispatcher

## Queuing Diagram



- When the OS creates a new process, it creates a PCB for the process and enters that process into the system in the **Not Running state**, and adds it into this Queue at the end.
- Now, the process exists, it is known to the OS, and is waiting for an opportunity to execute on the CPU.
- From time to time, the currently running process will be interrupted and the dispatcher portion of the OS would select some other process to run, when the currently running process completes running for a predefined **time quantum**.
- The former process moves from the Running state to the Not Running state, and the process which is at the head of the Queue moves to the Running state.

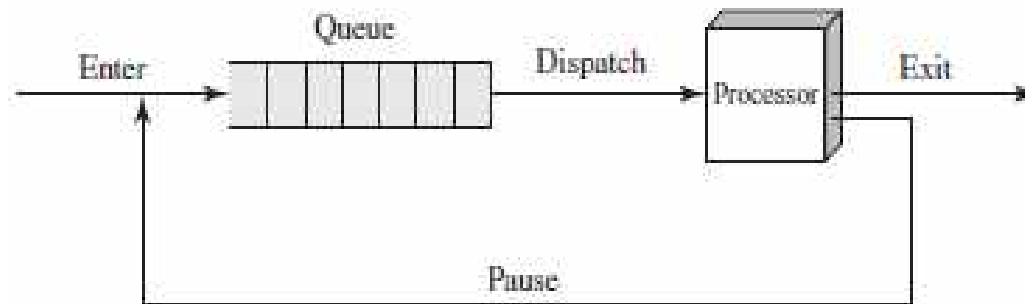
**Q1: Is this Queue FIFO or LIFO? FIFO**

**ANS: Pointers to the PCB structure of the processes waiting for the CPU.**

**Q1: What is stored in the queue of the Dispatcher?**



# Dispatcher Queue: Explained

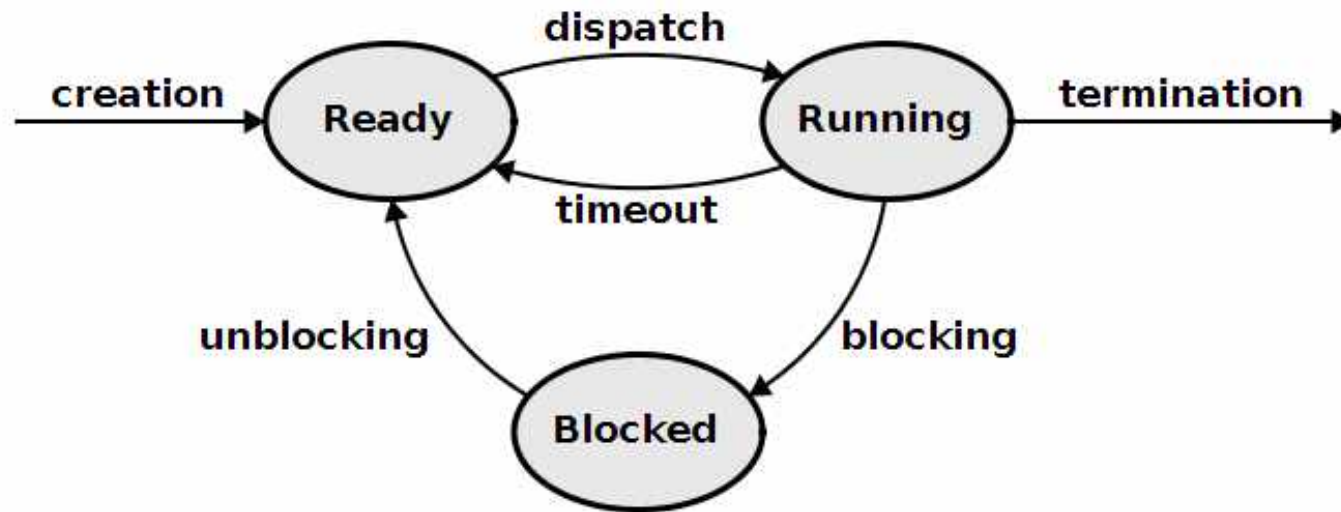


- The queue is a **first-in-first-out list** and the processor operates in **round-robin** fashion on the available processes (each process in the queue is given a **certain amount of time (quantum)**, in turn, to execute and then returned to the Ready or Dispatcher queue, unless blocked).
- However, even with the simple example that we have described, this implementation is inadequate: some processes in the Not Running state are ready to execute, while others are blocked, waiting for an I/O operation to complete.
- Thus, using a single queue, the dispatcher could not just select the process at the oldest end of the queue.
- Rather, the dispatcher would have to scan the list looking for the process that is not blocked and that has been in the queue the longest.

# OS Design Elements Related to Processes

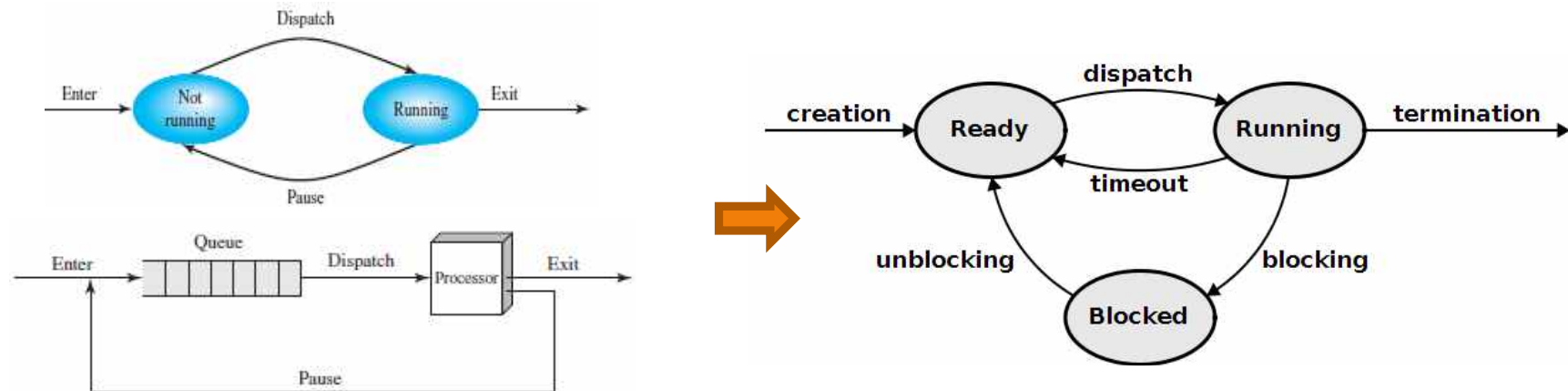
- From this simple model, we can already begin to appreciate some of the design elements of the OS.
- Each process must be represented in some way so that the OS can keep track of it.
- That is, there must be some information relating to each process maintained by the OS, including its current state and location in memory; in the process control block (PCB).
- Processes that are not running must be kept in some sort of queue, waiting their turn to execute.
- The structure of the OS Dispatcher suggests that there is a **queue** in which each entry is a **pointer to the process control block** of a particular process.
- Alternatively, the queue may consist of a **linked list** of data blocks, in which each block represents one process.

## Three States Model: Defined



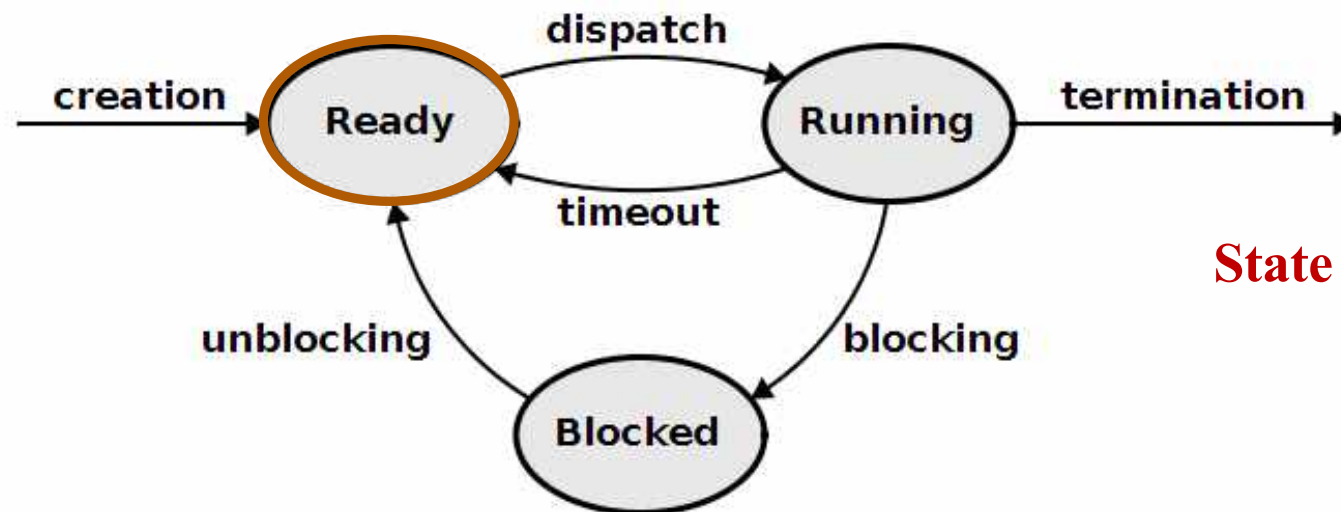
- The **state diagram** of a **process** captures its **life-cycle**.
- The states represent the execution status of the process; the transitions represent changes of its execution state.
- Each active process has its own execution status.
- Each process goes through their life-cycle, sequence of transitions, which may or may not depend on the life-cycle of other processes in the system.

# Three-State Process Model



- The two-state model has an inadequacy that since only two states are there, “Running” and “Not Running”
  - The processes which are blocked for I/O as well as the processes which are taken out of CPU, because of exceeding the time quantum, are both kept in the same Dispatcher queue.
- This makes the OS to go through this list multiple times, to choose the one which is not blocked on I/O to run on the CPU next,
- This is avoided by splitting the “Not Running” state into two states, “Blocked” and “Ready” states as shown above, separate Ready queue is maintained as well.

## Three-State Model Process: Ready State

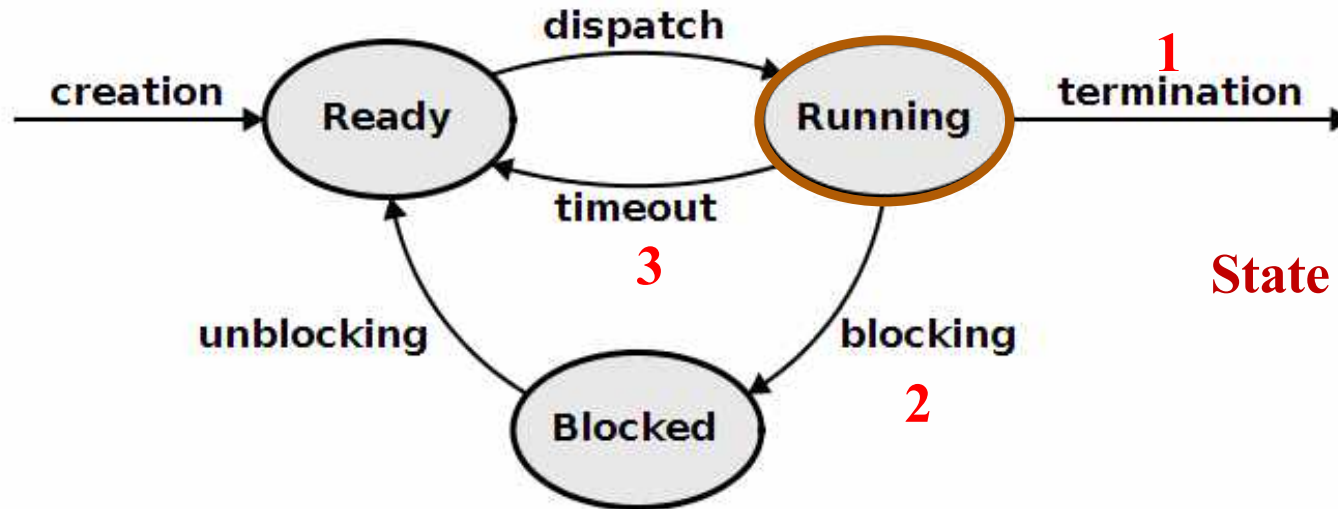


**State Diagram**

- A process in the **ready state** has all the resources that it needs for its execution, except for a processor.
- It is normally held in a ready queue until a processor becomes available.
- Here, the process is waiting to be assigned to a processor.
- Process may come into this state on creation or while running, it might have been interrupted by the scheduler, to assign the CPU to some other process.



# Three-State Model Process: Running State

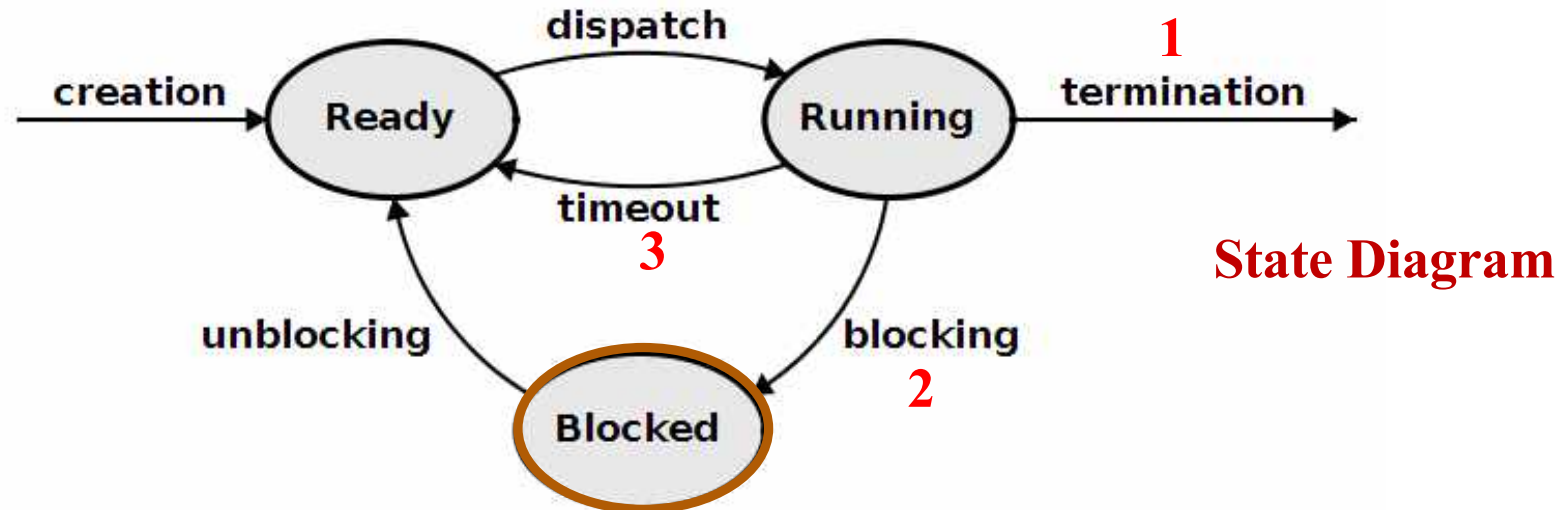


**State Diagram**

- A process in the running state has all of the resources that it needs for its execution, including the processor.
- It is currently running on the processor.
- A process can exit the running state for any of the following reasons:
  1. Process terminates or exits voluntarily by completing its job
  2. Process can go into Blocked state by initiating an I/O request.
  3. Process continues to run, exceeding its allotted time slot and the OS decides to forcibly take it away from the CPU to give a chance for some other process to run on the CPU which are waiting in the Ready queue.<sup>14</sup>

# Three-State Model Process:

## Blocked State



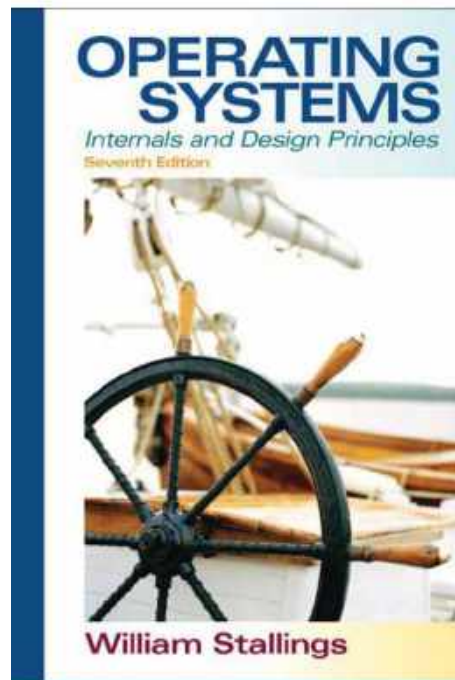
- A process that needs some resource other than the processor for further execution is in a blocked state.
- It is usually placed in a Blocked queue waiting for the needed resource.
- The needed resource could be a file to be opened from the HDD, or a char or data to be received from a serial port, or a mouse click from the user on the application window, etc.
- A process can stay in a blocked state for unlimited time because it depends on the occurrence of an external event.

## Session 2.2: Summary

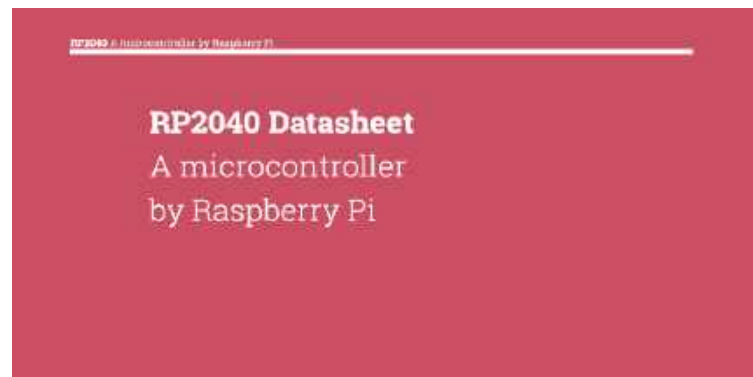
- **Process Control Block (PCB)**
- **Process State Models**
  - **Two States Model**
    - **Dispatcher Queue**
    - **Dispatcher Queue Explained**
  - **Three States Model**
    - **Ready**
    - **Running**
    - **Blocked**

# References (1 to 5)

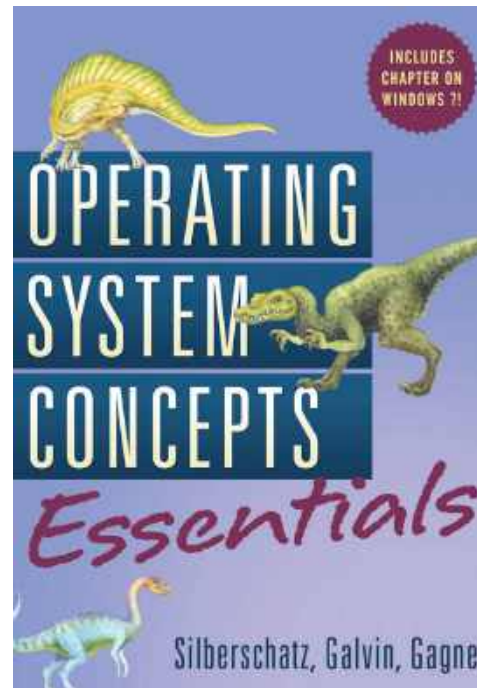
Ref 1



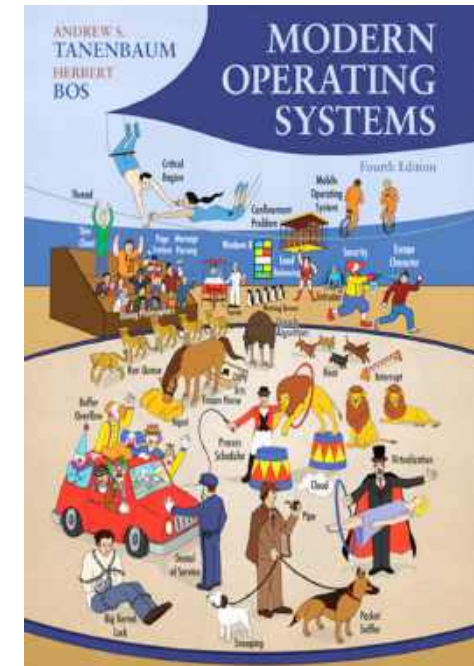
Ref 4



Ref 2



Ref 3



Ref 5

